

Project Report: AI Pathfinding Visualizer

VITyarthi Project 2025-26



VIT[®]
BHOPAL

NAME : NEHA SATISHKUMAR YADAV

REG ID : 25BCE11048

CSA2001 FUNDAMENTALS OF AI & ML

ACADEMIC YEAR: 2025-26 WINTER SEM

Index

Project Overview

Prerequisites

Project Details

Core Components

Algorithm Implementations

Execution & Controls

Conclusion

Future Improvements

Reference Links

This report details the development of an interactive AI Pathfinding Visualizer built using Python and Pygame. The application allows users to visualize how different search algorithms—Breadth-First Search (BFS), Depth-First Search (DFS), and A*—navigate through a grid-based environment to find the shortest path between two points.

1. Project Overview

The AI Pathfinding Visualizer is a tool designed to bridge the gap between theoretical algorithm knowledge and practical visualization. By providing a real-time, interactive grid, the project helps users understand the "cost" and "behaviour" of various search strategies. This project aligns with a focus on building a professional technical portfolio.

2. Prerequisites

To run or develop this project, the following environment setup is required:

Language: Python 3.x

Libraries: * pygame: For the graphical user interface and rendering.

collections: For efficient deque operations in BFS.

heapq: For priority queue management in the A* algorithm.

Knowledge Base: Understanding of graph theory, grid-based coordinate systems, and basic Data Structures (Stacks, Queues, and Priority Queues).

3. Project Details

Core Components

The project is structured around a Node class and a main simulation loop:

Node Class: Represents a single square in the 25×25 *grid*. It tracks its own coordinates, colour (state), and neighbours.

Grid System: A 2D list containing Node objects, where each node can be toggled between "Start," "End," "Wall," or "Empty."

State Visualization: Distinct colours are used to represent different node states:

-  Green: Start Point
-  Red: End Point
-  Black: Obstacles/Walls
-  Orange: Explored/Visited Nodes
-  Blue: Final Path

Algorithm Implementations

The visualizer includes three distinct algorithms, each with different performance characteristics:

1. Breadth-First Search (BFS):
 - Logic: Uses a Queue (FIFO) to explore nodes layer by layer.
 - Characteristic: Guarantees the shortest path in an unweighted grid but can be slow as it explores in all directions equally.
2. Depth-First Search (DFS):
 - Logic: Uses a Stack (LIFO) to explore as far as possible along a branch before backtracking.
 - Characteristic: Does not guarantee the shortest path and often creates "winding" paths.

A Search (A*):

Logic: Uses a Heuristic function $f(n) = g(n) + h(n)$, where $g(n)$ is the cost from start and $h(n)$ is the estimated cost to the end (Manhattan Distance).

Characteristic: Highly efficient and guarantees the shortest path by prioritizing nodes that appear closer to the target.

4. Execution & Controls

The user interacts with the visualizer using the mouse and keyboard:

1. Left Click: * 1st click: Set Start (Green)
 - 2nd click: Set End (Red)
 - Subsequent clicks: Draw Walls (Black)
2. Right Click: Reset a node to empty.
3. Keyboard Shortcuts:

B : Run BFS

D : Run DFS

A : Run A*

R : Reset the entire grid.

5. Conclusion

The project successfully demonstrates the efficiency of informed search (A*) compared to uninformed searches (BFS/DFS). Through this visualizer, one can clearly see how A* avoids unnecessary node exploration by using heuristics, making it the preferred choice for real-world navigation and gaming AI.

6. Future Improvements

Diagonal Movement: Update the `update_neighbors` method to allow 8-way movement.

Weighted Grids: Introduce "Mud" or "Water" nodes that have higher traversal costs to further showcase A* capabilities.

Dijkstra's Algorithm: Add Dijkstra as a middle ground between BFS and A*.

UI Enhancements: Add a sidebar for algorithm selection and a real-time counter for "Nodes Explored" and "Path Length."

7. Reference Links

Pygame Documentation: <https://www.pygame.org/docs/>

Introduction to A:* [Red Blob Games - Pathfinding](#)

Python heapq module: <https://docs.python.org/3/library/heapq.html>
